

ANALYSIS

GUI racefix execution gap — CLOSED, and what it was hiding

Field	Value
Revision	1
Created	2026-07-28
Last modified	2026-07-28
Status	active
Outcome	Gap CLOSED (not bounded); one real shipped defect found

Table of contents

- [Result](#)
- [Per-package build status](#)
- [Execution — all nine tests](#)
- [Defect found: 137 data races in shipped desktop chat](#)
- [Guard-strength finding — one mutation SURVIVED](#)
- [Secondary bluff surface](#)
- [Dependency list for the operator](#)
- [Honest gaps](#)

Result

Three `main_racefix_test.go` files (`desktop` / `harmony_os` / `aurora_os`) had been **authored but never executed** on this host. The recorded reason — “no X11/GL dev headers” — is **factually correct but not binding**: Fyne v2.7.0 ships a `ci` build tag that selects a non-GL driver.

All nine racefix tests now execute and PASS at `-race -count=3`. No dependency was installed; no production code changed to achieve it.

Why routine checks missed the gap: all three test files carry `//go:build !nogui`, and `make verify-compile-tests` runs `go test -tags=nogui -run='^$'` — so the files were never compiled, let alone run. The code under test (`updateLoopTick`, `startDataUpdates`, `Close/Cleanup`) lives entirely in each `main.go`, which is itself `!nogui`; there is no `nogui` counterpart to test instead.

Per-package build status

All three fail **identically** by default — the packages do not differ:

```
$ go vet ./applications/{desktop,harmony_os,aurora_os}/... # identical f
# [pkg-config --cflags -- gl gl] No package 'gl' found
# github.com/go-gl/glfw/v3.3/glfw
./glfw/src/x11_platform.h:33:10: fatal error: X11/Xlib.h: No such file or
```

Package	go vet default	go vet -tags=ci	racefix -race -count=3	full suite -race
desktop	FAIL (no X11/ GL)	PASS	3/3 PASS	FAIL — 137 data races
harmony_os	FAIL (no X11/ GL)	PASS	3/3 PASS	PASS
aurora_os	FAIL (no X11/ GL)	PASS	3/3 PASS	PASS

Probes — gl, x11, xrandr, xcursor, xinerama, xi, xxf86vm, xkbcommon, wayland-client, egl all MISSING; /usr/include/X11/Xlib.h and /usr/include/GL/gl.h MISSING; xvfb-run/Xvfb not on PATH. **Xvfb is irrelevant** — the failure is at COMPILE time, not runtime-for-want-of-a-display.

Execution — all nine tests

-tags=ci changes **only Fyne’s driver**. Verified our code is byte-identical under it (grep for a ci build tag across applications/ internal/ cmd/ → zero matches). The functions under test are pure channel/goroutine logic with no GUI calls, so this is full-strength evidence, not a weakened proxy.

```
ok dev.helix.code/applications/desktop      5.107s  EXIT=0  (x3, all PAS
ok dev.helix.code/applications/harmony_os    1.496s  EXIT=0  (x3, all PAS
ok dev.helix.code/applications/aurora_os     1.242s  EXIT=0  (x3, all PAS
```

No nil-deref, no leak, no hang. The 99ff7d8e defect class is **absent from these paths** — proven by execution, which beats the read-based review that was the fallback.

Defect found: 137 data races in shipped desktop chat

Running the newly-unlocked desktop suite under `-race` produced **137 WARNING: DATA RACE**. Exactly one production frame appears in all 137:

```
Write at 0x00c000402388 by goroutine 336:
  fyne.io/fyne/v2/widget.(*Entry).SetText()    entry.go:496
  dev.helix.code/applications/desktop.streamDesktopChat()
    applications/desktop/main.go:73
Previous read at 0x00c000402388 by goroutine 73:
  fyne.io/fyne/v2/widget.(*textRenderer).Layout() richtext.go:550
  fyne.io/fyne/v2/driver/software.RenderCanvas()  render.go:13
```

Root cause (§11.4.102, established before any fix). `helix_code/applications/desktop/main.go:57-64` asserts:

```
(*widget.Entry).SetText is goroutine-safe in Fyne, so
calling it from the caller's worker goroutine is correct.
```

That is FALSE for Fyne ≥ 2.6. Upstream `fyne.io/fyne/v2@v2.7.0/thread.go` states verbatim: *“DoAndWait is used to execute a specified function in the main Fyne runtime context. This is required when a background process wishes to adjust graphical elements of a running app. ... Since: 2.6”*.

`streamDesktopChat` is called at `main.go:1402` from inside a `go func(...)` worker (closed at `main.go:1408`) and mutates `da.chatHistory` directly — as do bare `SetText` calls at `main.go:1333, 1381, 1389, 1403, 1406`. And `grep -rn 'fyne\.Do|fyne\.DoAndWait' applications/` → **zero matches anywhere**.

Deliberately not fixed in the discovering pass.

`history.SetText(history.Text + content)` both reads and writes the widget, so a correct fix must move read *and* write inside one closure; `fyne.Do` is async while `DoAndWait` blocks, so the choice changes ordering/latency in the token-by-token streaming path this function exists to provide; it touches ≥6 call sites; and §11.4.125/§11.4.142 require independent review for a production concurrency change. The stale comment must be corrected in the same change.

Per §11.4.120: the desktop `-race` failure is the detector **correctly catching a real defect** and must NOT be suppressed. Without `-race`, desktop is 45/45 PASS.

Guard-strength finding — one mutation SURVIVED

Three paired §1.1 mutations on `harmony_os/main.go` `updateLoopTick`, each restored (sha256 identical, `git diff` empty, no marker residue):

Mutation	Guard	Result
A — remove priority pre-check only	StopAlwaysWins...	SURVIVED — test still PASSED
B — remove inner re-check only	ResidualWindow...	FAILED, 2497/5000 (~49.9%)
C — remove both (naked select)	StopAlwaysWins...	FAILED, 2559/5000 (~51.2%)

B and C confirm real teeth, matching the ~50% the fix commit predicted. But **A is a genuine guard gap**: with the pre-check deleted the inner re-check still catches the closed stop, so `tickWon` stays 0 and the test passes. A future refactor could delete half of the documented fix and **no test would notice**. Shipped code is correct; the *guard* is under-specified.

Secondary bluff surface

`TestRecordDesktopGUI` `LLMChat` made a real DeepSeek call that returned status 402: Insufficient Balance, and the test **still logged** `RECORD-OK: ...` real `deepseek/deepseek-v4-pro` reply captured. It certifies *bytes appeared in the widget*, not that the feature worked. It should assert the reply is not a provider-error shape and `SKIP-with-reason` (§11.4.3) when out of credit.

Dependency list for the operator

No installation is needed to run these tests — `-tags=ci` suffices and is the recommended headless path. Installation is only needed for the **real GLFW GUI binaries** (`make desktop / aurora-os / harmony-os`, which do not pass `-tags=ci`).

Host is ALT Workstation 11.1 (`apt-get + rpm`); runtime GL libs already present, only `-devel` missing. All confirmed available via read-only `apt-cache show`:

```
apt-get install -y libX11-devel libXcursor-devel libXrandr-devel libXinerama-devel  
libXi-devel libXxf86vm-devel libXext-devel libGL-devel libGLU-devel xorgproto-devel
```

Requires root — **deliberately not attempted**. Note `libGLX-devel` does not exist under that name here; `libGL-devel` provides the `gl.pc` that `go-gl/gl` looks for.

Honest gaps

- `-tags=ci` does not exercise the real **GLFW** driver. Immaterial for the nine racefix tests (no GUI calls); GLFW-specific rendering defects remain unmeasured on this host.
- **harmony_os** and **aurora_os** are NOT proven free of the same Fyne-threading defect — they pass only because neither has a GUI-record test driving a widget from a worker goroutine (5 and 6 `go func` blocks respectively in their `main.go`). Absence of a failing test is not absence of the defect (§11.4.118).
- The mutation battery ran on **harmony_os only**; the mutation-A guard gap is *expected* in the other two by structural identity, but was not separately measured.
- Single host, Go `go1.26.4-X:nodwarf5 linux/amd64`. No claim about `make desktop / aurora-os / harmony-os`, which were not built.